Project Report: Smart Farming System Using ML & DL

1. Project Overview

The **Smart Farming System** is a machine learning and deep learning-based application designed to assist farmers in making data-driven decisions for crop management. The system integrates **crop recommendation**, **disease detection**, and **smart agent functionalities** to optimize agricultural productivity.

Objectives:

- Predict the most suitable crop based on soil and climate parameters.
- Detect plant diseases from leaf images using a CNN model.
- Provide a unified intelligent interface to assist farmers.

2. System Components

2.1 Crop Predictor

The **Crop Predictor** module uses a trained ML model to recommend the best crop based on soil nutrients, environmental conditions, and climate data.

Key Features:

- Input Parameters:
    - N (Nitrogen), P (Phosphorus), K (Potassium)
    - Temperature, Humidity
    - pH, Rainfall
    - Soil Fertility
    - Climate Index
- Output: Recommended crop for optimal yield.

Implementation Details:

- Model Type: RandomForestClassifier (or equivalent ML model)
- Preprocessing: Data is scaled using StandardScaler.

- Label Encoding: Crops are encoded using LabelEncoder.
- Libraries: joblib (for model persistence), numpy.

Sample Usage:

```
features = {
  'N': 90, 'P': 42, 'K': 43,
  'temperature': 25, 'humidity': 80,
  'ph': 6.5, 'rainfall': 200,
  'soil_fertility': 7, 'climate_index': 0.85
}
crop_predictor = CropPredictor()
recommended_crop = crop_predictor.predict(features)
```

2.2 Disease Detector

The **Disease Detector** module uses a Convolutional Neural Network (CNN) to detect diseases in plants from leaf images.

Key Features:

- Input: Leaf image of the plant.
- Output: Disease classification.
- Model Format: TensorFlow/Keras CNN model (.h5 or SavedModel format).

Implementation Details:

- The model is fine-tuned on pre-trained CNN architectures.
- Images are preprocessed to match the input shape of the CNN.
- Libraries: tensorflow, keras, h5py.

Usage Example:

```
disease_detector = DiseaseDetector()
result = disease_detector.predict("path_to_leaf_image.jpg")
```

Notes:

- Models trained in Google Colab have been converted for local use.
- Conversion to TensorFlow SavedModel ensures compatibility with local TensorFlow 2.x environments.

2.3 Smart Farming Agent

The **Smart Farming Agent** integrates all modules into a single intelligent interface.

Key Features:

- Uses Crop Predictor and Disease Detector.
- Provides automated recommendations for crop selection and disease management.
- Supports additional tools (e.g., RAG QA tool for knowledge queries).

Implementation Details:

```
class SmartFarmingAgent:
  def __init__(self):
    self.crop_tool = CropPredictor()
    self.disease_tool = DiseaseDetector()
```

- Can be extended to include:
    - Pest prediction
    - Irrigation scheduling
    - Soil health monitoring

3. System Architecture

1. **Data Collection**: Soil data, environmental parameters, crop databases.
2. **Preprocessing**: Scaling, label encoding, image resizing.
3. **ML/DL Models**:

    - Crop Predictor (Random Forest / Decision Tree)
    - Disease Detector (CNN)
4. **Smart Agent**: Combines predictions and recommendations.
5. **User Interface** (Optional): Web or desktop interface for farmers.

4. Technologies and Libraries Used

- **Programming Language**: Python 3.13
- **Machine Learning**: scikit-learn, joblib, numpy
- **Deep Learning**: tensorflow, keras
- **Image Handling**: Pillow, OpenCV
- **Model Persistence**: .pkl (joblib), .h5 (Keras), SavedModel
- **Additional Tools**: sentence-transformers, faiss (for RAG QA)

5. Challenges and Solutions

- **Model Loading Errors**: Legacy .h5 files trained in Colab caused "bad object header" errors locally. Solved by converting models to TensorFlow SavedModel format.
- **Library Version Compatibility**: Ensured all models work with TensorFlow 2.12 and compatible h5py.
- **Module Import Issues**: Resolved Python path and package visibility problems by structuring the src folder properly and using relative imports where needed.

6. Results

- Crop Predictor successfully recommends crops with reasonable accuracy using soil and climate inputs.
- Disease Detector identifies leaf diseases effectively, demonstrating the power of CNNs for image classification.
- Smart Farming Agent integrates both models, providing a unified interface for farmer decision-making.

7. Future Work

- Expand dataset to include more crop types and diseases.
- Implement a real-time web interface for farmers.
- Add IoT-based sensors for live soil and weather data input.
- Improve CNN architecture for better disease classification accuracy.

8. Conclusion

This project demonstrates the practical application of **Machine Learning** and **Deep Learning** in agriculture. It provides farmers with actionable insights, enabling data-driven decisions for crop management and disease prevention.